

Practical 1: Music simulator

Introduction

Go to your preferred large language model (LLM), such as DeepSeek or ChatGPT, and ask it: ‘Generate eight measures of Baroque music in the style of Vivaldi. Give me the score using scientific pitch notation, encoding note durations as lowercase letters. Provide only a single verbatim output, stating the time and key signatures (place a hash at the start of these lines), and remove all measure titles.’ It may respond with something like this:

```
# Time signature: 4/4
# Key signature: G major
A4e D5e F#5e A5e G5e F#5e E5e D5e |
F#5q E5e D5e Re A4e D5q |
E5e F#5e G5e A5e B5e A5e G5e F#5e |
E5q C#5e D5e F#5e A5e G5q |
F#5e E5e D5e C#5e B4e C#5e D5e E5e |
F#5q Rq A4e B4e C#5q |
D5e F#5e A5e F#5e E5e D5e C#5e B4e |
A4h D5q Rq ||
```

For those who are familiar with music theory and Western music notation, the above text encodes a short piece of music written G major in the $\frac{4}{4}$ time signature. Each row represents a single measure (or bar) consisting of a sequence of symbols encoding a note. A note is written in the form [NoteName][Octave][Duration], encoding a note’s pitch and duration. For example, A4h is a half note (aka minim) at A above middle C, F#5e is an eighth-note (aka quaver) at F# above A4, and Rq is a rest for the duration of a quarter-note (aka crotchet). The music score for the above generated output is presented in Figure 1.



Figure 1: Music score of generated music in the Baroque style by ChatGPT.

For those who are not familiar with music theory, consider each ‘note’ (e.g. A4e) as a word, each line is a statement or sentence, and the whole output as a paragraph or poem. As with language, music has patterns and structure that can be used to predict the next note in order to replicate a particular style or composer. AI models are trained on huge quantities of data to predict the next word, note or value to produce an output in response to a user’s prompt. Highly specialised and well trained AI models can produce output where it is difficult to distinguish between a simulation and what it is designed to mimic (<https://www.youtube.com/shorts/8IYGdahQpbE>).

This practical is about coding up what you might term a simple ‘small prediction model’ to simulate music that resembles the training data: Renaissance choral music by Giovanni Palestrina, accessed from <https://music21.org> and provided by the supplementary text document `palestrina.txt`.

Details of the prediction model

We will work with a Markov model. Given a sequence of p notes, the model will randomly select the next note depending on the probabilities that each possible note follows the given sequence. These probabilities are computed

from the scores of choral music written in text form using scientific pitch notation, as illustrated above. Typically we would start with a single note, generate the next note from the model and then use the resulting 2 note sequence to generate the next note, then the 3 note sequence to generate the next note until we have an m note sequence, where m is the maximum order of model considered. Given an m note sequence we then predict the next note. To predict each note thereafter, we use the sequence of the most recent m note generated. For example, below presents an example of the initial steps for generating a note sequence for a 3rd order model starting at G5e using the above ChatCPT repose as reference:

```
step 1) Input: G5e -> Prediction: F#5e
step 2) Input: G5e F#5e -> Prediction: E5e
step 3) Input: G5e F#5e E5e -> Prediction: D5e
step 4) Input: F#5e E5e D5e -> Prediction: C#5e
```

The overall sequence G5e F#5e E5e D5e C#5e does not appear in the reference data.

How exactly should the probabilities for predicting the next note be obtained from the text? For each m note sequence we could simply locate each time the sequence occurs in the text and tabulate the note that follow it, and their frequency. For example, the sequence F#5e E5e D5e in the reference data is followed by C#5e twice and a end-of-measure return character | (aka a bar-line) once. Hence the next note is predicted to be either C#5e with probability 0.667 or | with probability 0.333, with probabilities for all other notes in the text set to 0.

How large should m be? There is a trade-off here. Using a large m gives lots of ‘context’ from which to predict the next note, encapsulating features such as the number of notes per measure and stylistic features of the composer. However, a large m typically leads to only a small (or no) occurrences of certain note sequence in the text - so we get a rather poor noisy estimate of the likely distribution for the next note. One way to deal with this is to predict the next note from a mixture of next note distributions generated by the last m notes, the last $m - 1$ notes and so on down to the last note. For example, the frequencies for the next note based on the sequence F#5e E5e D5e and its subsequences within the reference data is:

```
F#5e E5e D5e -> 2x C#5e, 1x |
E5e D5e -> 2x C#5e, 1x |, 1x Re
D5e -> 3x F#5e, 2x C#5e, 1x |, 1x Re, 1x E5e
```

This gives a nice balance between the richer context provided by larger note sequences and the richer data available for shorter ones.

To be more mathematically concrete, let $\mathbf{v}$ be the vector containing the latest sequence of m notes. Then we generate the next note from the distribution

$$\sum_{i=1}^m w_i P(\text{next note} | \mathbf{v}[i : m]) \quad (1)$$

where the w_i are mixture weights summing to 1 (e.g. $w_i = 1/m$ by default).

To make this work well requires some simplification. We will not work with the raw string encoding of a note, but with numerical ‘tokens’ representing them. The tokens will be the integers giving the location of the note in the vector of unique notes. For example, if $\mathbf{b}$ is the vector of k unique note encodings within the reference data and $b_9 = \text{F\#5e}$ then the token for the note F#5e is 9.

How best to store the probabilities in (1) and sample notes/tokens according to those probabilities? The obvious approach creates a $k \times k \times \dots \times k$ array ($m+1$ dimensions) containing the probabilities of each common note token following each possible common note token sequence of length m . We then work through the text adding one to the array entry indexed by each length $m+1$ token sequence encountered. At the end we normalize the entries sharing the same initial m tokens (where these sum to > 0) to obtain the probability estimates¹. The problem is that this approach would require storing an enormous array, that your computer is unlikely to manage, where the majority of the array entries would be zero! All far too inefficient to use.

Instead, if n is the number of notes in the text, we can create a matrix, $\mathbf{M}$ with $n - m$ rows and $m + 1$ columns, where the rows contain all the sequences of $m + 1$ note tokens occurring in the text. This will let us sample according to the model probabilities ‘on the fly’. Here is how...

Let $\mathbf{v}$ be an m note token sequence. We first find all the rows of $\mathbf{M}$ whose first m elements match $\mathbf{v}$. Now collect the $m + 1$ th element of each of those rows into one vector $\mathbf{u}$, and associate with each element of $\mathbf{u}$ a probability

¹we would need to repeat this for $m - 1, m - 2$ etc order models

w_m/n_u where n_u is the number of elements in $\mathbf{u}$ and w_m the mixture weight. Together $\mathbf{u}$ and the probability constitute an empirical representation of the distribution of notes that might follow $\mathbf{v}$ in the text. Typically, notes may occur in $\mathbf{u}$ several times.

We can now drop the first element of $\mathbf{v}$ and the first column of $\mathbf{M}$ and repeat the process to get the distribution of notes likely to follow $\mathbf{v}[2:m]$. Similarly the distributions of notes likely to follow $\mathbf{v}[3:m]$ etc can be obtained. The combined $\mathbf{u}$ vectors and corresponding probabilities then represent an approximation to the distribution in (1). We can sample from this distribution by simply concatenating the $\mathbf{u}$ vectors, and the corresponding vectors of probabilities, and using the `sample` function to sample one token from this distribution. In the event that the combined $\mathbf{u}$ vector contains no elements, we may as well select a common note at random from the text (i.e. sample a common note according to its occurrence probability in the text).

Task

Your task is to write code implementing this simple prediction model and simulate music using it. Because this is the first practical, the instructions for how to produce code will be unusually detailed: the task has been broken down for you. Obviously the process of breaking down a task into constituent parts before coding is part of programming, so in future practicals you should expect less of this detailed specification.

As a group of 3, collaborating using `git`, you should aim to produce well commented², clear and efficient code implementing the model and simulating music with it. The code should be written in a plain text file called `proj1.r` and is what you will submit. Your solutions should use only the functions available in base R. The work must be completed in your work group of 3, which you must have arranged and registered on Learn. The first comment in your code should list the names and university user names of each member of the group. The second comment **must** give a brief description of what each team member contributed to the project, and roughly what proportion of the work was undertaken by each team member. Contributions never end up completely equal, but you should aim for rough equality, with team members each making sure to ‘pull their weight’, as well as not unfairly dominating³.

Note that `unlist` can be quite useful for some of the tasks below, and that you will almost certainly progress much faster if you bother to learn how to use the `debug` package to `mtrace` your functions.

1. Create a repo for this project on github, and clone to your local machines.
2. Download from Learn the plain text file `palestrina.txt` containing all of the music scores composed by Giovanni Palestrina using scientific pitch notation.
3. The following code will read the file into R. You will need to change the path in the `setwd` call to point to your local repo. Do not change the name of the text file! This is to facilitate marking.

```
setwd("put/your/local/repo/location/here") ## comment out of submitted
music_raw <- readLines("palestrina.txt")
```

Check the help file for any function whose purpose you are unclear of. The `setwd` line should be commented out of the code when you finally submit your work for marking.

4. Use the `head` command to look at the first 10-20 lines of `music_raw` to examine what has been loaded. Some pre-processing of `music_raw` is needed before using the data in a prediction model. Write a function called `preprocess` that takes the vector argument `music` as input, and performs the following tasks:
 - (a) Remove the first three lines containing details about the composer and the data source.

²Good comments give an overview of what the code does, as well as line-by-line information to help the reader understand the code. Generally the code file should start with an overview of what the code in that file is about, and a high level outline of what it is doing. Similarly each function should start with a description of its inputs outputs and purpose plus a brief outline of how it works. Line-by-line comments aim to make the code easier to understand in detail. The comments should not be mechanical descriptions of what lines of R code do. The aim is to explain what is being done and why, in a way that helps the reader understand the logic and purpose behind the code.

³all team members must have `git` installed and use it - not doing so will count against you if there are problems of seriously unequal contributions

- (b) All of the lines *beginning* with # provides supplementary information about the music, such as the voice, time signature, key signature, etc. These lines need to be removed. Note that the character # is also used to denote sharp notes (e.g. F#4q), so be careful with how you determine which lines to remove.
- (c) The file contains multiple music scripts, with a single blank line separating each piece. It is important to keep these pieces separate for the prediction model as it would be inappropriate to use note sequences that span across two scores. To indicate the end of a piece, at the end of the last measure (line) add the characters " ||" (space-pipe-pipe) to represent a double bar-line as illustrated at the end of Figure 1. In natural language, consider this like a full-stop.
- (d) Each line of notes represent a measure (bar) of music. Add at the end of these lines (except for the last measure in a piece) the characters " |" (space-pipe) to indicate a bar-line. In natural language, consider these as commas.
- (e) Finally, split each measure into individual notes, rests and bar-line symbols (henceforth collectively called 'notes'). You may use the fact that each note is separated by a space character.

If performed correctly, the length of the returned vector should be over a million, with the last 10 entries being:

```
music_clean <- preprocess(music = music_raw)
tail(music_clean, n = 10)
[1] "D3h" "C3h" "G3w" "|" "D3b" "|" "G3b" "|" "G3b" "||"
```

5. The function `unique` constructs a vector, `notes`, of unique notes in the (cleaned) text, `music_clean`. The function `match` can then be used to find the index of which element in `notes` each element of `music_clean` corresponds to. Here's a small example illustrating `match`.

```
match(x = c("Re", "A4e", "A4e", "A4e", "F4h", "|", "Re", "G4e", "G4e", "G4e", "E4h", "|"),
      table = c("Re", "A4e", "F4h", "|", "G4e", "E4h"))
[1] 1 2 2 2 3 4 1 5 5 5 6 4
```

- (a) Use `unique` to make the vector `notes`, the vector of unique notes.
 - (b) Use `match` to make the index vector `tokens` that determines for each element in the text corresponds to the index position within `notes`. The corresponding index vector should be the same length as `music_clean`.
6. Now you need to make the matrices of common note token sequences. Create a second function called `make_matrix` that has 3 input arguments: `tokens`, the integer vector of tokens; `end`; the token ID for the double bar-line 'note' `||`; and, `mlag`, the maximum lag considered (i.e. m above). Your code should be written to work with any reasonable value of `mlag`, but set a default value of 4. To construct the matrix M , this function must:

- (a) Create an $(n - \text{mlag}) \times (\text{mlag} + 1)$ matrix, M , where n is the length of `tokens` vector. The first column should contain the token vector excluding the last `mlag` elements. The second column is the next token for each following notes, i.e. the token vector excluding the first element and the last `mlag-1` elements. The next column is shifted once more again, and so on. The final `mlag+1` column should contain the token vector excluding the first `mlag` elements. Each row of M gives the indices in `tokens` of every sequence of `mlag+1` adjacent notes in the text.
- (b) Currently, M contains rows where the token ID for double bar-line, `||`, appears *within* sequences. This 'note' is only permitted at the end of a sequence, else otherwise the sequence spans across two pieces of music. Remove from M all rows where `end` (the token ID for `||`) appears anywhere within the first `mlag` columns.

If working correctly, the following code should return the sequence matrix of tokens.

```
M <- make_matrix(tokens = tokens, end = which(notes=="||"), mlag = 4)
```

7. Now write a function called `next_note` that samples the next note from a given sequence. The function should have three inputs: `key`, a vector containing a token sequence for which the next note is to be generated; `M`, the matrix produced by `make_matrix`; and, `w` a vector of mixture weights (which actually don't need to be normalized). A default value for `w` should be specified as a vector of 1s (see the command `rep`) of length `mlag` (determine this length based on the argument `M`).

The function should return a token for the next note, generated according to the model described above. It should be able to deal appropriately with any length of `key`: using reduced order versions of the model for short keys, and using only data from the end of `key` if it is too long.

The crucial part of the function is (repeatedly) finding the rows of `M` that match `key` (or its reduced versions). Suppose the current `key` is to be matched to columns `mc:mlag` of `M`. Now compute

```
ii <- colSums(! (t(M[,mc:mlag,drop=FALSE]) == key))
```

If the j th element, `ii[j]`, is zero then row j of `M` contains a match.

To simulate the next note, your function should perform at most one call to the `sample` command based on the number of found matches to the token sequence and sub-sequences:

- If no matches are found or the probability associated to all found matches is zero (e.g. due to zero-valued lagged weights), then any token should be sampled from the final column of `M`.
 - If exactly one match has been found, return *the* next token.
 - If there are two or more matches, then sample the next note based on the probabilities estimated by (1).
8. Finally, write the function called `generate_music` to simulate from the model. The inputs to this function must be: `start`, a single token ID to initiate the simulation; `M`, the matrix generated by `make_matrix`; `w`, a vector of mixing weights (specify default weights); and, `notes` the vector of note names. The function should repeatedly call `next_note`, using the generated token sequence (starting with `start`) as input for `key`. The code should continue to generate music until one of the following stopping conditions have been met:

- The double bar-line `||` has been generated, indicating the end of the piece;
- Eight measures (bars) have been generated (once `|` has been generated for the eighth time); or
- The generated piece of music consists of 200 notes (including bar-lines).

The token IDs for the relevant note should be obtained from `notes`.

Convert the generated tokens back to the note text, and paste the vector into a single string (see the `paste` command) with notes separated by a space. Your `generate_music` function should return a single long string of characters representing your generate music, as demonstrated below

```
sim_music <- generate_music(start = 20, M = M, notes = notes,
                             w = c(1, 1, 1, 1))

sim_music
[1] "F#5w Rh D5h | B3h C4h | C4w Rh F3h | F3w C3w | ...."
```

Be aware that the model is 'simple' and determining a consistent note duration across all measures may not always be possible.

Test that your function works for different weights. For each of the weight vectors $w_1 = (1, 1, 1, 1)$, $w_2 = (1, 0, 0, 0)$ and $w_3 = (0, 0, 0, 1)$, briefly comment on what type of lagged dependence the prediction model is using, and how this impacts what you see in the structure of music generated by `generate_music`.

On Learn is the R script `play_music.r` that contains functions that can convert the a single string containing a piece of music into a waveform which can be played using the "audio" package.

```
## install.packages("audio") ## Run this line ONCE to download the package
source("play_music.r")
play(str_to_waveform(sim_music, bpm = 200))
```

Do not include the above code in your workflow.

Submission

One piece of work - the text file containing your commented R code - is to be submitted for each group of 3 on Learn platform. You may be asked to supply an invitation to your github repo, so ensure this is in good order. No extensions are available on this course, because of the frequency with which work has to be submitted. So late work will automatically attract a penalty (of 100% after work has been marked and returned). Technology failures will not be accepted as a reason for lateness (unless it is provably the case that university systems were unavailable for an extended period), so aim to submit ahead of time.

Marking Scheme: Full marks will be obtained for code that:

1. does what it is supposed to do, and has been coded in R approximately as indicated using base R (that is marks will be lost for simply finding a package or online code that simplifies the task for you).
2. is carefully commented, so that someone reviewing the code can easily tell exactly what it is for, what it is doing and how it is doing it without having read this sheet, or knowing anything else about the code. Note that *easily tell* implies that the comments must also be as clear and *concise* as possible. You should assume that the reader knows basic R, but not that they know exactly what every function in R does.
3. is well structured and laid out, so that the code itself, and its underlying logic, are easy to follow.
4. is reasonably efficient. As a rough guide the whole code to generate a note sequence should take at most 10 seconds to run - much longer than that and something is probably wrong.
5. was prepared collaboratively using git and github in a group of 3.
6. contains no evidence of having been copied, in whole or in part, from anyone else (AI counts as anyone else for this purpose), other students on this course, students at other universities (there are now tools to help detect this, including internationally), online sources, ChatGPT etc.
7. includes the comment stating team member contributions.